

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management

Systems COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyse DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Concept Mapping (Medium)

Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Construct a concept map linking the following terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.	BL2 – Understand
2	Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.	BL2 – Understand
3	Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.	BL2 – Understand
4	Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.	BL2 – Understand
5	Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.	BL2 – Understand
6	Develop a concept map for the EER model showing how it extends the basic ER model with generalization, specialization, and aggregation.	BL2 – Understand
7	Create a visual concept map for the DBMS software architecture, illustrating the flow from user request to data retrieval.	BL2 – Understand
8	Map the data models discussed in CO1 to real-world applications and explain the suitability of each model.	BL2 – Understand
9	Construct a concept map for schemas and instances, showing how they relate to physical and logical data independence.	BL2 – Understand

10	Design a concept map for the roles and responsibilities in a DBMS environment: DBA, End User, Application Developer.	BL1 – Remember
----	--	----------------

Code of Conduct:

I **ADHAVAN CHANDRASEKAR(192521209)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **ADHAVAN CHANDRASEKAR(192521209)**

Concept Map 1: Core DBMS Terminology

Construct a concept map linking: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.

Overview

Definition

- DBMS (Database Management System) is software that defines, creates, maintains and controls access to a database.
- Schema is the overall logical design/structure of the database, defined at the time of database design.
- Instance is the actual data stored in the database at a specific moment in time (a snapshot).
- Data Model is a set of concepts/notations used to describe the structure of a database (e.g., ER model, relational model).
- ER Diagram is a graphical representation of entities, attributes and relationships produced using the ER data model.
- Entity is a real-world object/concept that has independent existence and is distinguishable from other objects (e.g., Student).
- Attribute is a property or characteristic that describes an entity (e.g., Name, RollNo).
- Relationship is an association among two or more entities (e.g., Student ENROLLS_IN Course).

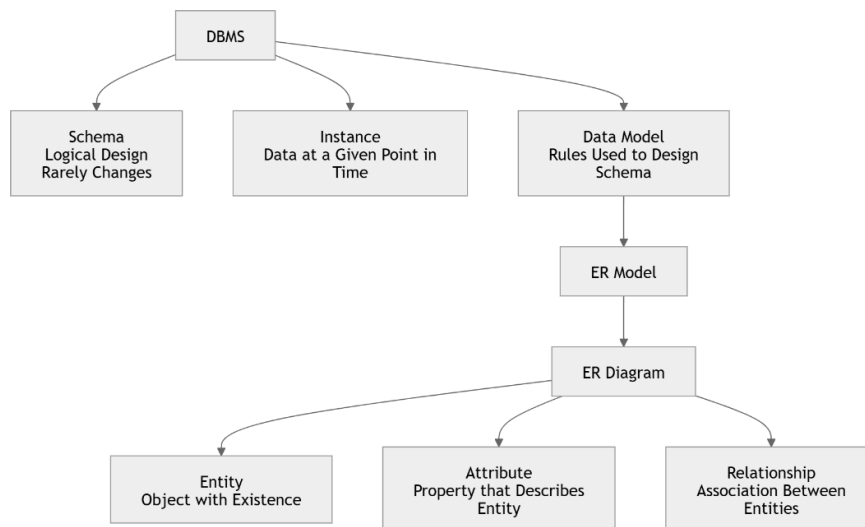
Purpose

- Shows how the foundational vocabulary of DBMS is interconnected rather than isolated definitions.
- Helps learners see that a Data Model is the language used to build an ER Diagram, which in turn is translated into a Schema.

Key Features

- DBMS is the umbrella system that manages Schema, Instance and enforces the Data Model.
- Schema is stable/rarely changed; Instance changes every time data is inserted, updated or deleted.
- ER Diagram is a visual tool built from Entities, Attributes and Relationships, following a chosen Data Model.

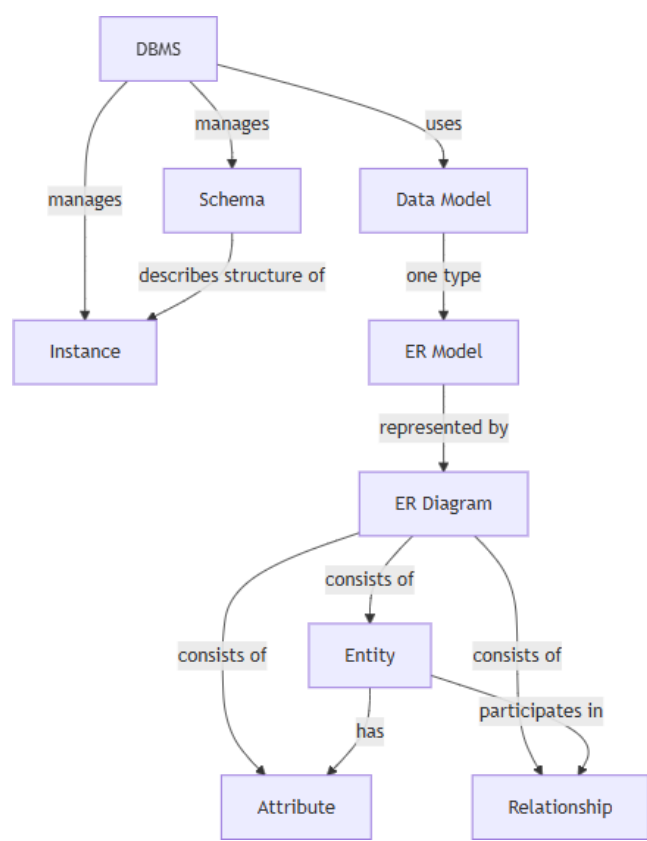
Concept Hierarchy



Components

Component	Description
DBMS	Software layer that manages data, schema, instances and user access.
Schema	Blueprint/structure of the database (tables, fields, constraints).
Instance	Actual content of the database at one moment; changes over time.
Data Model	Conceptual toolset (e.g., ER, Relational) to describe schema.
ER Diagram	Diagrammatic output of applying the ER data model.
Entity	A distinguishable real-world object represented in the schema.
Attribute	A named property that characterizes an entity.
Relationship	A meaningful association connecting two or more entities.

Relationship



Examples

- DBMS example: MySQL, Oracle, PostgreSQL.
- Schema example: CREATE TABLE Student(RollNo, Name, Dept) definition.
- Instance example: the 500 current student rows stored today.
- ER Diagram example: Student — ENROLLS_IN — Course diagram.

Summary

- DBMS is the managing software; Schema is the design; Instance is the live data.
- Data Model is the conceptual language; ER Diagram is its graphical output.
- Entities, Attributes and Relationships are the building blocks that make up an ER Diagram and, ultimately, the Schema.

Concept Map 2: Three-Level ANSI/SPARC Architecture

Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.

Overview

Definition

- The ANSI/SPARC architecture divides a database system into three levels: External, Conceptual and Internal, to separate user views from physical storage.

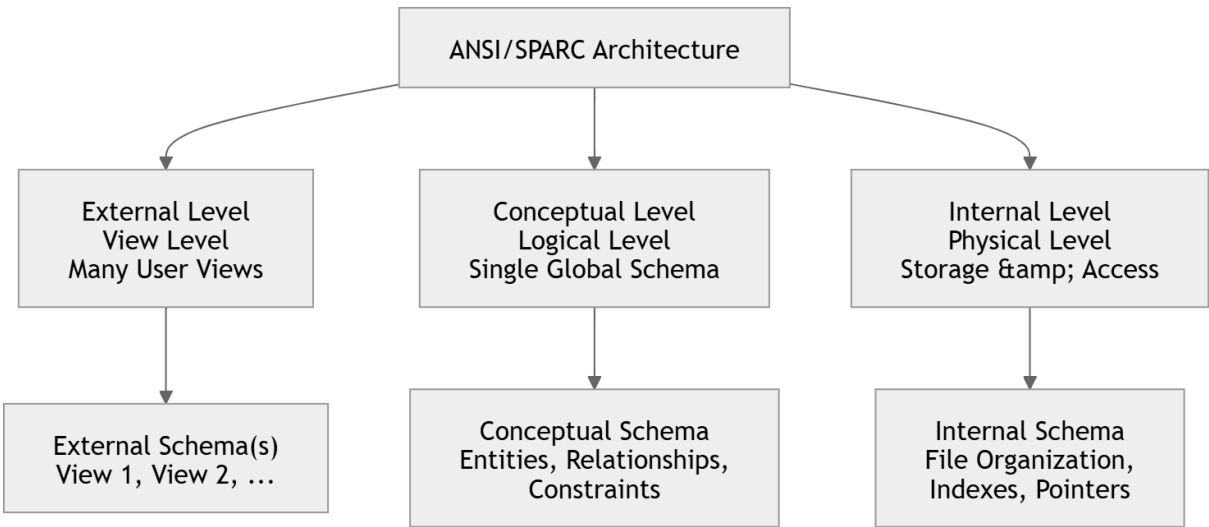
Purpose

- Achieves data abstraction so users are shielded from storage details.
- Provides data independence so changes at one level do not force changes at another.

Key Features

- External Level: multiple user/application-specific views of the database.
- Conceptual Level: single, community/global logical structure of the whole database.
- Internal Level: physical storage structures, indexes, file organization.
- Mappings exist between External↔Conceptual and Conceptual↔Internal levels.

Concept Hierarchy

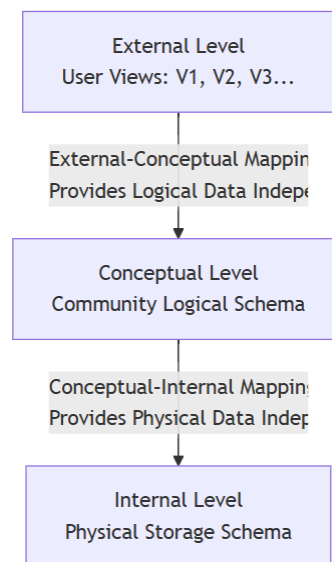


Components

Component	Description
External Level	Defines how individual users see the data (custom views).
Conceptual Level	Community view; describes entities, relationships, constraints for the whole enterprise.
Component	Description

Internal Level	Describes physical storage: indexing, file structures, access paths.
Logical Data Independence	Ability to change conceptual schema without changing external schemas.
Physical Data Independence	Ability to change internal schema without changing conceptual schema.

Multi-Level Architecture



Quick Reference

Term	Meaning
View Level	Another name for the External Level.
Logical Level	Another name for the Conceptual Level.
Physical Level	Another name for the Internal Level.
Mapping	Rules that translate a request from one level to the level below it.

Examples

- External: a payroll clerk's view showing only salary-related columns.
- Conceptual: full Employee table with all attributes and relationships.
- Internal: B-tree index on EmployeeID stored on disk.

Summary

- Three levels separate concerns: what the user sees, what the whole system logically contains, and how it is physically stored.
- Logical data independence protects external views from conceptual schema changes.
- Physical data independence protects the conceptual schema from internal storage changes.

Concept Map 3: File-Based Systems vs Database Systems

Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.

Overview

Definition

- A file-based system stores data in separate flat files managed directly by application programs.
- A database system centralises data storage and access through a DBMS layer shared by all applications.

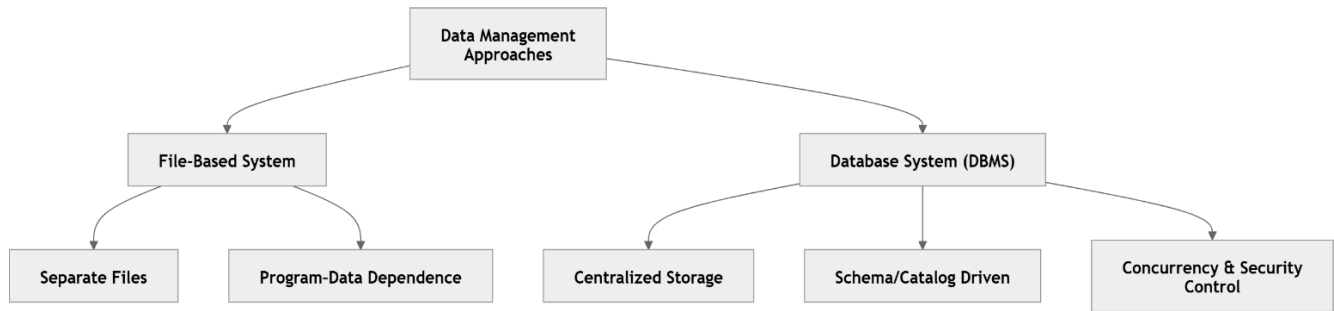
Purpose

- Illustrates why database systems replaced file-based systems by resolving the latter's structural weaknesses.

Key Features

- File systems: data and program are tightly coupled; each application maintains its own files.
- Database systems: data is centralised, self-described (via schema/catalog) and shared across applications.

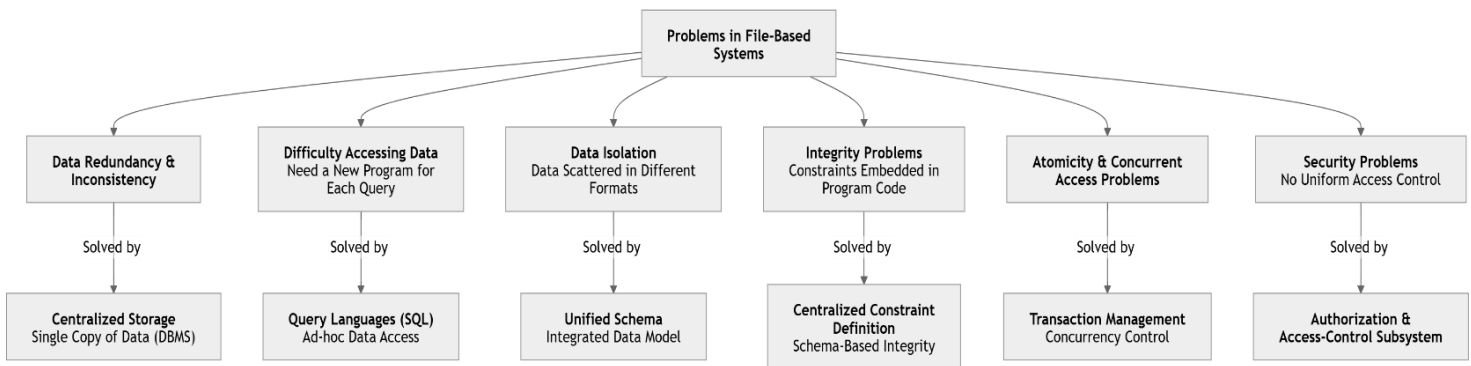
Concept Hierarchy



Comparison

Feature	File-Based System	Database System (DBMS)
Data Redundancy	High – same data duplicated across files	Minimised via normalisation & central storage
Data Sharing	Difficult; program-specific files	Easy; shared through DBMS
Data Independence	None; programs depend on file structure	Provided (logical & physical)
Integrity Constraints	Enforced in application code	Enforced centrally in schema
Concurrent Access	Not well supported	Managed via concurrency control
Security	Weak, per-file	Centralised authorisation

Problem → Solution Mapping



Examples

- File-based: COBOL payroll program reading its own EMP.DAT file.
- Database: an ERP system where Sales, HR and Finance modules all query one shared database via a DBMS.

Summary

- File-based systems suffer from redundancy, inconsistency, isolation, poor security and weak concurrency handling.
- DBMS solves these by centralising data, enforcing schema-driven integrity, and providing transaction, security and query support.

Concept Map 4: ER Diagram Components

Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.

Overview

Definition

- An ER Diagram graphically models data requirements using entities, attributes, relationships and constraints.

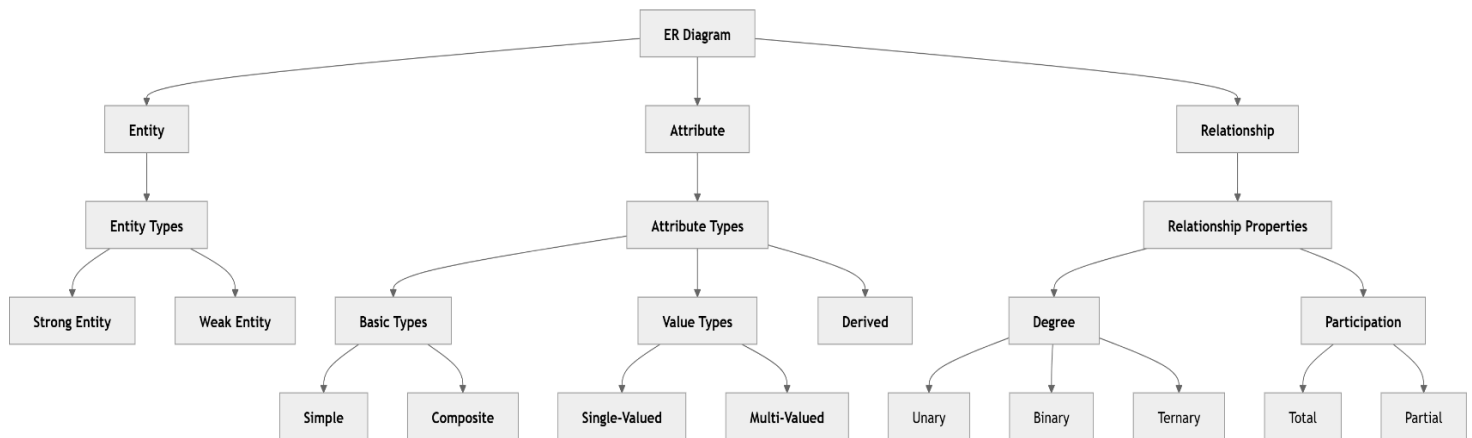
Purpose

- Breaks down every notational building block of the ER model so relationships between the components are visible at a glance.

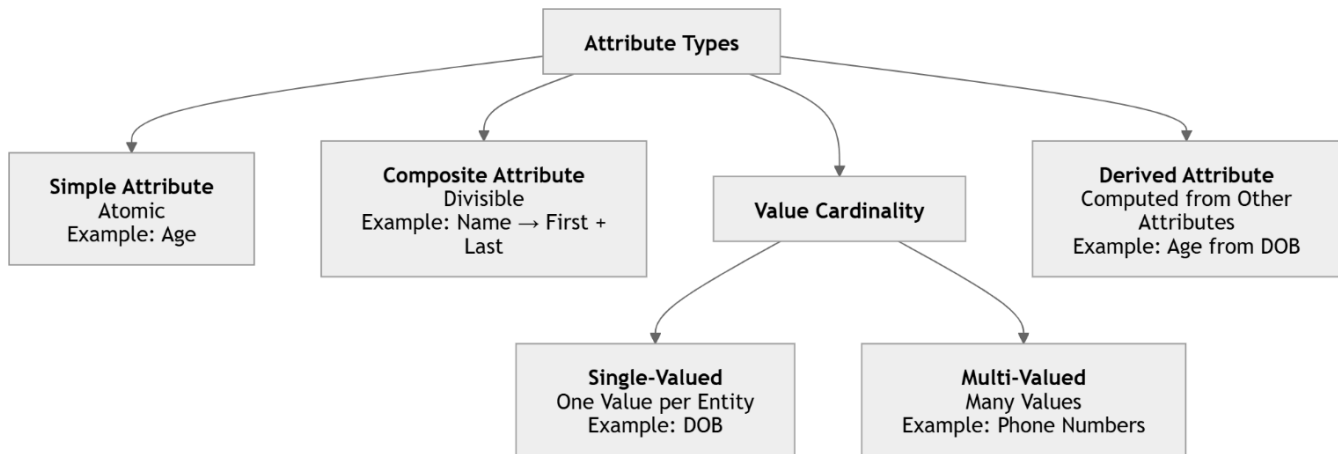
Key Features

- Entities can be strong (independent) or weak (dependent on an owner entity).
- Attributes may be simple, composite, single-valued, multi-valued or derived.
- Relationships carry a degree (unary, binary, ternary) and cardinality (1:1, 1:N, M:N).
- Participation constraints specify whether an entity's involvement in a relationship is total or partial.

Concept Hierarchy



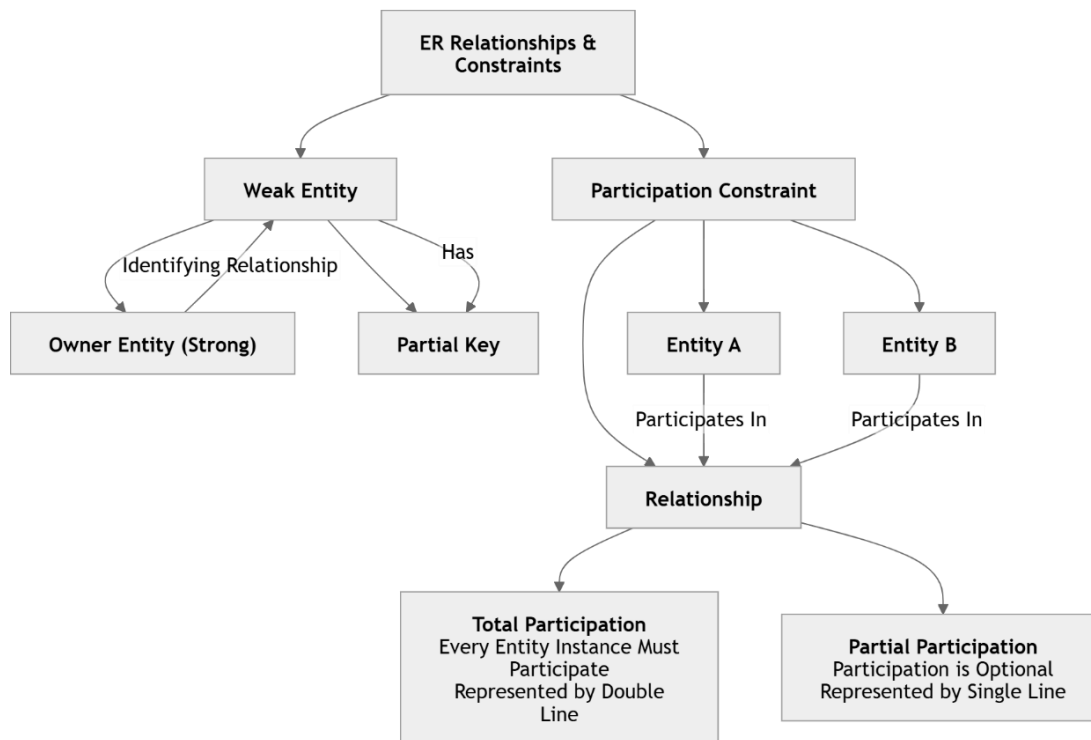
Types / Classification



Components

Component	Description
Strong Entity	Has its own primary key; exists independently.
Weak Entity	Has no primary key of its own; depends on an owner (identifying) entity, uses a partial key.
Simple Attribute	Cannot be divided further, e.g. Age.
Composite Attribute	Can be divided into sub-parts, e.g. Name = First + Last.
Multi-valued Attribute	Can hold multiple values, e.g. Phone Numbers.
Derived Attribute	Computed from another attribute, e.g. Age from DateOfBirth.
Relationship Degree	Number of entities participating: unary, binary, ternary.
Participation Constraint	Total (double line, mandatory) or Partial (single line, optional).

Relationship



Quick Reference

Term	Meaning
Double rectangle	Weak entity
Double diamond	Identifying relationship
Dashed underline	Derived attribute
Double line	Total participation
Oval with lines	Multi-valued attribute

Examples

- Strong entity: Employee (PK: EmployeeID).
- Weak entity: Dependent (owned by Employee, partial key: DependentName).
- Total participation: every Loan must be associated with a Bank (mandatory).

Summary

- Entities may be strong or weak; weak entities rely on an identifying relationship with an owner.
- Attributes vary in nature: simple, composite, multi-valued, derived.
- Relationships are described by degree and constrained by cardinality and participation (total/partial).

Concept Map 5: Hierarchical, Network and Relational Data Models

Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.

Overview

Definition

- A data model is a collection of concepts used to describe the structure, operations and constraints of a database.
- Hierarchical, Network and Relational are three classical (legacy-to-modern) data models.

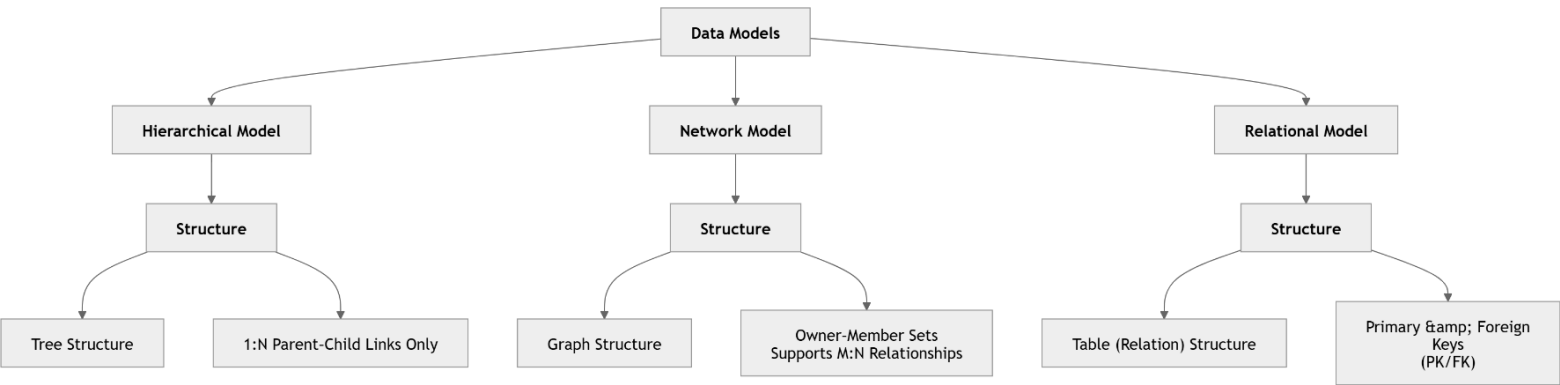
Purpose

- Contrasts the structural philosophy (tree, graph, table) of each model to show their evolution and trade-offs.

Key Features

- Hierarchical Model: tree structure with parent-child (1:N) links only.
- Network Model: graph structure allowing many-to-many relationships via owner-member sets.
- Relational Model: data organised into tables (relations) linked by keys; based on set theory.

Concept Hierarchy



Components

Component	Description
Hierarchical Model	Records organised as a tree; each child has exactly one parent.
Network Model	Records organised as a graph; a member can have multiple owners via sets.
Relational Model	Data stored as rows/columns in tables; relationships via primary/foreign keys.

Comparison

Property	Hierarchical	Network	Relational
Structure	Tree (one root)	Graph (multiple parents)	Set of tables
Relationship type	1:N only	1:1, 1:N, M:N	1:1, 1:N, M:N (via keys)
Navigation	Top-down traversal	Pointer/set traversal	Declarative (SQL)
Flexibility	Rigid	Complex but flexible	Highly flexible
Redundancy	Can be high	Moderate	Reduced via normalisation
Example System	IMS (IBM)	IDMS	MySQL, Oracle, PostgreSQL

Examples

- Hierarchical: an organisation chart stored as IMS records (Company → Department → Employee).
- Network: a university system where a Student can belong to multiple Clubs and vice versa.
- Relational: Student table linked to Course table via an Enrollment table using foreign keys.

Summary

- Hierarchical model is simplest but restricted to strict 1:N tree relationships.
- Network model relaxes this to support M:N via owner-member sets, at the cost of navigational complexity.
- Relational model replaces navigation with declarative, key-based access, offering the most flexibility and is the current industry standard.